claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-a9f26048adf11a2f9.jsonl`
- SHA-256: `97ea631d702b09aede23e95a1584aeb2d1a517aa40d1a60c9a457f64809d5a99`
- Source modified: `2026-06-22T15:07:05+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_f39953fd-98d`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T15:05:03.847Z)

Adversarially verify this finding by reading the cited code in C:/Users/avidu/Projects/bhi-cloudrun (run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD`). Real defect to fix, or false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: dispatch-correctness
Severity: major
Title: returncode!=0 branch is effectively dead: worker writes no failure marker and hardcodes rc=0, so cloud job failures surface only as a poll timeout
File: backend/main.py:515-516
Detail: The gate `if result.returncode != 0: return _failed(...)` can essentially never fire for a real worker-side failure. In backend/worker/__main__.py the done-marker is written ONLY on the success path (line 259-260) and with a HARDCODED returncode of 0 (`_write_done_marker(args.done_uri, video_id, 0, ...)`). Validation failure returns 2 (line 200) and segmenter Failure returns 3 (line 212) BEFORE the marker is ever written, so no marker is produced. CloudRunCompute.run_infer (backend/compute/cloud_run.py:119) then bucket-polls forever and poll_until raises PolicyTimeout (it is NOT caught/converted to rc=4 there, despite the base.py:37 docstring claiming '4 = remote dispatch timeout' ? that rc is never produced). The PolicyTimeout is caught by the broad except at main.py:512-514 and reported as a generic 'cloud dispatch error', after the FULL max_wait_sec elapses. So a bad-video validation failure on the cloud path costs a multi-minute hang + an opaque error instead of a fast, accurate 'failed validation' ? and the explicit returncode!=0 handling is unreachable for genuine failures.
Fix: Make the worker write a done-marker on its failure paths too (rc=2/3) before returning, and pass the real returncode into _write_done_marker instead of the hardcoded 0; then the main.py rc!=0 gate becomes meaningful and failures are fast + accurately messaged. Alternatively, at minimum, fix the base.py/cloud_run.py docstrings to stop claiming an rc=4 path that doesn't exist.

2. assistant (2026-06-22T15:05:06.218Z)

I'll verify this finding by examining the cited code in the bhi-cloudrun repo.

3. user (2026-06-22T15:05:07.290Z)

```
 backend/main.py                  | 100 ++++++++++++++++++
 tests/test_api_cloud_dispatch.py | 212 ++++++++++++++++++++++++++++++++++++++++
 2 files changed, 312 insertions(+)
```

4. assistant (2026-06-22T15:05:09.140Z)

Let me look at the diff and the actual files cited.

5. user (2026-06-22T15:05:10.223Z)

304252b feat(serving): wire /api/process to dispatch DETECT to Cloud Run (M6 -> API), default-OFF

6. user (2026-06-22T15:05:12.808Z)

```
450         local = storage.fetch(uri, dst, config=storage_config)
451         n = 0
452         with open(local, newline="", encoding="utf-8") as f:
453             for row in csv.DictReader(f):
454                 try:
455                     start, end = float(row["start"]), float(row["end"])
456                 except (KeyError, TypeError, ValueError):
457                     continue
458                 meta: Dict[str, Any] = {"source": "cloud_run",
459                                         "ending_reason": (row.get("ending_reason") or
"").strip()}
460                 sc = (row.get("shot_count") or "").strip()
461                 if sc:
462                     try:
463                         meta["shot_count"] = int(float(sc))
464                     except ValueError:
465                         meta["shot_count"] = sc
466                 db_instance.add_play_segment(video_id, start, end, metadata=meta)
467                 n += 1
468         return n
469
470
471     def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str,
val_results,
472                                play_wm: int, cand_wm: int, notify) -> Optional[Dict[str,
Any]]:
473         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
474         == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
475         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
476
477         Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
``None`` to
478         fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend``
returns ``None``
479         for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
480         from backend.compute import ComputeJobSpec, get_compute_backend
481
482         compute = get_compute_backend(global_config)
483         if compute is None:
484             return None  # default-OFF ? run the in-process segmenter below (unchanged)
485
486         storage_cfg = getattr(global_config, "storage", None)
487
488         def _failed(msg: str) -> Dict[str, Any]:
489             # Shape-match the in-process segmenter-fail branch + restore prior good rows
(none written yet).
490             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
491             return {"video_id": video_id, "status": "failed", "message": msg,
492                     "results": [r.model_dump() for r in val_results], "play_segments": []}
493
494         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
495         try:
496             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
497         except ValueError as e:
498             return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
499         if input_ref.backend == "local":
500             return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
```

```
this machine "
501                             "can't be read by the Cloud Run job. Put the video in your bucket
first.")
502         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
503         if not out_root:
504             return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
505                             "? that's where the Cloud Run job writes the result.")
506
507         notify("dispatching (cloud_run)")
508         logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
509         spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name)
510         try:
511             result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
512         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
513             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
514             return _failed(f"cloud dispatch error: {e}")
515         if result.returncode != 0:
516             return _failed(f"cloud job failed (returncode {result.returncode}).")
517
518         notify("ingesting (cloud_run)")
519         storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
{}
520         _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
521         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
522         segments = [s for s in db_instance.query_play_segments(video_id, {}) if
int(s.get("id", 0)) > play_wm]
523         _finalize_reingest(video_id, segments, play_wm, cand_wm)
524         return {"video_id": video_id, "status": "success",
525                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
526                 "results": [r.model_dump() for r in val_results], "play_segments": segments}
527
528
529     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
530         """The full hash ? validate ? segment ? report pipeline for one video.
531
532         This is the former synchronous /api/process body, unchanged in behavior, now run on
533         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
534         the sync endpoint used to return (UI compatibility); the per-video observability
535         report is still emitted in `finally:` and grafted as response["reports"].
536
537         `progress` is an optional callable(stage: str) used to surface coarse job progress.
538         Raises on unexpected internal errors ? the caller records those as a job failure
539         (after this function has already restored the pre-run segments, see below).
540         """
541         notify = progress or (lambda stage: None)
542
543         notify("hashing")
544         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
545         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
546         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
547         # consumers (hash / validators / segmenter) use the resolved local path.
548         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
549         video_id = compute_video_id(local_video)
550         was_reingest = db_instance.get_video(video_id) is not None
```

```
551
552         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
553         notify("validating")
554         logger.info(f"Running validations for {req.video_path}...")
555         val_results, val_context = registry.run_all_with_context(local_video, global_config)
556         failures = [r for r in val_results if not r.passed]
557
558         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
559         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
560         seg_name = req.segmenter or default_seg
561         segmenter_actual = None
562         segmentation_ran = False
563         response: Optional[Dict[str, Any]] = None
564         try:
565             if failures:
566                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
567                 # status; it no longer destroys the video's prior good segments.
568                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
569                 response = {
```

7. assistant (2026-06-22T15:05:15.597Z)

The main.py side is confirmed. Now let me verify the worker side and the compute backend. These files are NOT in the diff (only main.py and the test changed), so I need to read the existing files on the branch.

8. user (2026-06-22T15:05:16.605Z)

```
180         print(f"[worker] pulled {args.video_uri} -> {local_video}")
181
182         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
183         video_id = compute_video_id(local_video)
184
185         # 3. VALIDATE (same registry as the main CLI).
186         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
187         failures = [r for r in val_results if not r.passed]
188         db = Database(os.path.join(scratch, config.output.db_name))
189         db.add_video(video_id, local_video, "failed" if failures else "processed",
190                      [r.model_dump() for r in val_results])
191
192         segments: List[dict] = []
193         segmenter_actual = None
194         segmentation_ran = False
195         status = "failed"
196         try:
197             if failures:
198                 print("[worker] validation FAILED: "
199                       + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
200                 return 2
201             seg_name = args.segmenter or config.indexing.default_segmenter
202             segmenter_cls = segmenter_registry.get(seg_name)
203             if not segmenter_cls:
204                 print(f"[worker] unknown segmenter: {seg_name!r} "
205                       f"(have {list(segmenter_registry.list_available())})")
206                 return 2
207             segmenter_actual = seg_name
208             result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
209             segmentation_ran = True
210             if isinstance(result, Failure):
```

```
211                 print(f"[worker] segmenter FAILED: {result.message}")
212                 return 3
213             segments = result.value if hasattr(result, "value") else result
214             status = "success"
215             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
216             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
217             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
218             if not segments:
219                 detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
220                                  or getattr(config.indexing, "detector_impl", None) or
"auto")
221                 logger.warning(
222                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
223                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
224                     "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
225                     "the WASB env produced no detections, or the input resolution differs
from the tuned "
226                     "proxy resolution. Re-run with -v for the native command + frame
counts.",
227                     video_id, segmenter_actual, detector_impl,
228                 )
229         finally:
230             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
231             emit_indexer_report(
232                 db=db, video_id=video_id, video_path=local_video, config=config,
233                 val_results=val_results, val_context=val_context,
234                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
235                 was_reingest=False, segmentation_ran=segmentation_ran,
236             )
237
238         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
239         out_local = os.path.join(scratch, "outputs", video_id)
240         os.makedirs(out_local, exist_ok=True)
241         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
242         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
243
244         out_prefix = args.out_uri.rstrip("/")
245         pushed = []
246         for fn in sorted(os.listdir(out_local)):
247             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
248             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
249             pushed.append(uri)
250
251         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
252         for u in pushed:
253             print("   ", u)
254
255         # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
256         # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
257         # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
258         # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
259         if getattr(args, "done_uri", None):
```

```
260             _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
261         return 0
262
263
264     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
265     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
266     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
267
268     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
269     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
```

9. assistant (2026-06-22T15:05:25.914Z)

The worker side is confirmed exactly as described:
- Line 200: validation failure returns 2 (before any marker)
- Line 212: segmenter Failure returns 3 (before any marker)
- Lines 259-260: done-marker written ONLY on the success path, with hardcoded `0`

The comment at lines 256-258 even explicitly acknowledges this: "Written on SUCCESS only ? a failed container job currently lets the dispatcher's bounded poll time out (a failure-marker is a follow-up)."

Now let me verify the cloud_run.py poll behavior and the base.py docstring claim.

10. user (2026-06-22T15:05:26.866Z)

```
90              self._client = run_client
91              self._job_name = job_name
92              self._region = region
93              self._project = project
94              self._storage_config = storage_config or {}
95              self._policy = policy
96              # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97              # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98              self._token = token
99              self._container_overrides = (
100                 _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101             )
102
103         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104             out_root = spec.out_uri.rstrip("/")
105             token = self._token or uuid.uuid4().hex
106             done_uri = f"{out_root}/_dispatch/{token}.done"
107             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                                "--done-uri", done_uri]
109             if spec.segmenter:
110                 argv += ["--segmenter", spec.segmenter]
111             for kv in (*spec.config_overrides, *self._container_overrides):
112                 argv += ["--set", kv]
113
114             execution = self._client.run_job(self._job_name, argv, region=self._region,
115                                              project=self._project)
116             LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117                      self._job_name, execution, done_uri)
118
119             marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
```

```
120                                 label="cloud-run-infer", logger=LOG)
121             video_id = str(marker.get("video_id", ""))
122             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124             rc = int(marker.get("returncode", 1))
125             if not video_id:
126                 LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                             "video_id) ? treating as failure", self._job_name)
128                 rc = rc or 1
129             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131             LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132             return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134         # --- storage-backed completion signal (bucket-poll) ---
135         def _read_marker(self, done_uri: str) -> Optional[dict]:
136             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137             ref = parse_uri(done_uri)
138             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139                 return None
140             return self._read_json(done_uri) or {}  # empty dict still terminates the poll
(present == done)
141
142         def _read_json(self, uri: str) -> dict:
143             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146             try:
147                 ref = parse_uri(uri)
148                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149                 local = fetch(uri, dst, config=self._storage_config)
150                 with open(local, "r", encoding="utf-8") as f:
151                     data: Any = json.load(f)
152                 return data if isinstance(data, dict) else {}
153             except Exception:
154                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155                 return {}
156
```

11. assistant (2026-06-22T15:05:30.220Z)

Confirmed. `poll_until` (line 119) waits for the marker. Since the worker only writes the marker on success, a validation/segmenter failure produces no marker, so `poll_until` will run until the policy timeout. Let me verify what `poll_until` does on timeout, and the base.py docstring.

12. user (2026-06-22T15:05:30.910Z)

```
1       """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3       COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4       (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5       core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6       (cloud-serving) ? without the core ever branching on the profile (D1).
7
```

```
8       The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9       non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10      existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11      """
12
13      from __future__ import annotations
14
15      from abc import ABC, abstractmethod
16      from dataclasses import dataclass, field
17      from typing import Optional
18
19
20      @dataclass(frozen=True)
21      class ComputeJobSpec:
22          """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23          from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24          deterministic regardless of where the job ran."""
25
26          video_uri: str                          # input (gs://? for cloud; a path/local://
for local)
27          out_uri: str                            # output ROOT (gs://? bucket for cloud)
28          segmenter: Optional[str] = None
29          # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30          config_overrides: tuple[str, ...] = ()
31
32
33      @dataclass(frozen=True)
34      class ComputeResult:
35          """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36          (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37          empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38          done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40          returncode: int
41          outputs_uri: str = ""
42          video_id: str = ""
43          report: dict = field(default_factory=dict)
44
45
46      class ComputeBackend(ABC):
47          """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48          its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49          execution identically."""
50
51          @abstractmethod
52          def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53              """Run ``spec`` to completion and return its ``ComputeResult``."""
54
```

13. assistant (2026-06-22T15:05:35.172Z)

The base.py docstring (lines 36-38) confirms the finding's claim: it documents "4 = remote dispatch timeout or an empty/unreadable completion marker (the cloud path only)". Now let me verify whether `poll_until` actually raises (rather than returning rc=4) on timeout, and whether

anything catches PolicyTimeout and converts it.

14. user (2026-06-22T15:05:37.321Z)

```
backend/compute/cloud_run.py:7:  2. **Bucket-poll** ``done_uri`` via
``execution_policy.poll_until`` + the storage layer (D16 ? the
backend/compute/cloud_run.py:33:from backend.infrastructure.execution_policy import
DEFAULT_POLICY, ExecutionPolicy, poll_until
backend/compute/cloud_run.py:119:        marker = poll_until(lambda:
self._read_marker(done_uri), self._policy,
backend/compute/cloud_run.py:136:        """The done-marker dict if the worker has written it,
else None ? poll_until keeps waiting."""
backend/compute/cloud_run.py:7:  2. **Bucket-poll** ``done_uri`` via
``execution_policy.poll_until`` + the storage layer (D16 ? the
backend/compute/cloud_run.py:33:from backend.infrastructure.execution_policy import
DEFAULT_POLICY, ExecutionPolicy, poll_until
backend/compute/cloud_run.py:119:        marker = poll_until(lambda:
self._read_marker(done_uri), self._policy,
backend/compute/cloud_run.py:136:        """The done-marker dict if the worker has written it,
else None ? poll_until keeps waiting."""
backend/infrastructure/execution_policy.py:7:- ``poll_until`` ? a bounded poll loop for an in-
progress external op (``poll_every_n_sec`` cadence,
backend/infrastructure/execution_policy.py:39:class PolicyTimeout(TimeoutError):
backend/infrastructure/execution_policy.py:83:    Returns ``fn``'s value on success. Raises
``PolicyTimeout`` if the final attempt hung, or re-raises the
backend/infrastructure/execution_policy.py:113:            last_exc = PolicyTimeout(f"{label}
hung > op_timeout_sec={policy.op_timeout_sec}s")
backend/infrastructure/execution_policy.py:125:def poll_until(check: Callable[[], Optional[T]],
policy: ExecutionPolicy = DEFAULT_POLICY, *,
backend/infrastructure/execution_policy.py:130:    ``PolicyTimeout`` after ``max_wait_sec``.
``check`` returns ``None`` while the op is still in progress.
backend/infrastructure/execution_policy.py:147:            raise PolicyTimeout(f"{label} not
ready within max_wait_sec={policy.max_wait_sec}s")
backend/providers/gemini.py:10:    ExecutionPolicy, PolicyTimeout, poll_until, run_bounded,
backend/providers/gemini.py:90:            video_file = poll_until(_processed, process_policy,
backend/providers/gemini.py:92:        except PolicyTimeout:
backend/providers/gemini.py:129:        except Exception as e:                  # PolicyTimeout
(hang) or the last 503 ? both end here
backend/worker/__main__.py:124:    poll exhaust its budget ? ``PolicyTimeout``. Catch it and
return a CLEAN rc 4 (remote dispatch
backend/worker/__main__.py:126:    from backend.infrastructure.execution_policy import
PolicyTimeout
backend/worker/__main__.py:134:    except PolicyTimeout as e:
```

15. assistant (2026-06-22T15:05:41.069Z)

Interesting. There's a `PolicyTimeout` catch in worker/__main__.py around line 124-134 that mentions "return a CLEAN rc 4". Let me examine that ? it may be a separate code path (the train/distributed path, not infer dispatch). Let me read it.

16. user (2026-06-22T15:05:42.314Z)

```
115             return False  # already logged at ERROR by enforce_startup_checks
116
117
118     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
119         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
120         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
121         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
122
123         A remote job that never writes a completion marker (a failed/hung container) makes
```

```
the bucket
124         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
125         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
126         from backend.infrastructure.execution_policy import PolicyTimeout
127
128         spec = ComputeJobSpec(
129             video_uri=args.video_uri, out_uri=args.out_uri,
130             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
131         )
132         try:
133             result = compute.run_infer(spec)
134         except PolicyTimeout as e:
135             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
136             return 4
137         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
138                     result.video_id, result.returncode, result.outputs_uri)
139         return result.returncode
140
141
142     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
143                            status: str, storage_cfg: dict, scratch: str) -> None:
144         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
145         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
146         try:
147             marker = {"video_id": video_id, "returncode": returncode,
148                       "segment_count": segment_count, "status": status}
149             local = os.path.join(scratch, "_done.json")
150             with open(local, "w", encoding="utf-8") as f:
151                 json.dump(marker, f)
152             storage.put(local, done_uri, config=storage_cfg)
153             logger.info("[worker] wrote completion marker -> %s", done_uri)
154         except Exception as e:  # never fail a good run because the marker push hiccuped
155             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
156
157
158     def cmd_infer(args: argparse.Namespace) -> int:
159         config = load_config(args.config) if args.config else load_config()
```

17. assistant (2026-06-22T15:05:48.964Z)

This is an important nuance. There IS a `_dispatch_remote` function (lines 118-139) that catches `PolicyTimeout` and returns rc=4. This is the WORKER-side CLI dispatch path (`backend.worker` invoked with `compute_target=cloud_run`).

But the finding is specifically about the **API path in main.py** (`_maybe_dispatch_remote`), which calls `compute.run_infer(spec)` **directly** at line 511 ? NOT through `_dispatch_remote`. So the rc=4 conversion in the worker CLI does NOT apply to the main.py API path. The finding correctly scopes its claim to `main.py:512-514` catching the raw `PolicyTimeout` via the broad except.

Let me confirm: does main.py's `_maybe_dispatch_remote` go through `_dispatch_remote` or call `run_infer` directly? Line 511 clearly calls `compute.run_infer(spec)` directly. So the rc=4 path exists ONLY in the worker CLI, not in the API path. Let me verify `poll_until` raises PolicyTimeout (line 147) and confirm the policy timeout magnitude.

18. user (2026-06-22T15:05:49.914Z)

```
125     def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
```

```
126                     label: str = "op", logger: Optional[logging.Logger] = None,
127                     sleep: Callable[[float], None] = _time.sleep,
128                     monotonic: Callable[[], float] = _time.monotonic) -> T:
129         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
130         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
131
132         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133         """
134         log = logger or LOG
135         start = monotonic()
136         polls = 0
137         log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
138                  label, policy.poll_every_n_sec, policy.max_wait_sec)
139         while True:
140             result = check()
141             if result is not None:
142                 log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
start)
143                 return result
144             elapsed = monotonic() - start
145             if elapsed >= policy.max_wait_sec:
146                 log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,
elapsed)
147                 raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
151
```

19. assistant (2026-06-22T15:05:53.587Z)

Confirmed: `poll_until` raises `PolicyTimeout` on timeout (line 147). It does not return rc=4 itself.

Now let me check the `_policy` passed to the CloudRunCompute in the API path to understand the `max_wait_sec` magnitude (to validate the "multi-minute hang" claim), and verify there's no rc=4 conversion in the main.py API path's exception handling.

20. user (2026-06-22T15:05:55.467Z)

```
backend/infrastructure/execution_policy.py:8:  ``max_wait_sec`` deadline).
backend/infrastructure/execution_policy.py:40:    """An op hung past ``op_timeout_sec`` or never
became ready within ``max_wait_sec``."""
backend/infrastructure/execution_policy.py:49:    max_wait_sec: float = 1800.0          # total
wall budget for a poll loop
backend/infrastructure/execution_policy.py:70:DEFAULT_POLICY = ExecutionPolicy()
backend/infrastructure/execution_policy.py:78:def run_bounded(fn: Callable[[], T], policy:
ExecutionPolicy = DEFAULT_POLICY, *,
backend/infrastructure/execution_policy.py:125:def poll_until(check: Callable[[], Optional[T]],
policy: ExecutionPolicy = DEFAULT_POLICY, *,
backend/infrastructure/execution_policy.py:130:    ``PolicyTimeout`` after ``max_wait_sec``.
``check`` returns ``None`` while the op is still in progress.
backend/infrastructure/execution_policy.py:138:             label, policy.poll_every_n_sec,
policy.max_wait_sec)
backend/infrastructure/execution_policy.py:145:        if elapsed >= policy.max_wait_sec:
backend/infrastructure/execution_policy.py:147:            raise PolicyTimeout(f"{label} not
ready within max_wait_sec={policy.max_wait_sec}s")
```

21. assistant (2026-06-22T15:05:59.012Z)

The default `max_wait_sec` is 1800.0 (30 minutes). Let me check how the factory constructs the policy for the API-path CloudRunCompute, to confirm the wait magnitude that the API path would experience.

22. user (2026-06-22T15:06:00.557Z)

```
Exit code 2
backend/compute/base.py
backend/compute/cloud_run.py
backend/compute/__init__.py
---FACTORY---
backend/compute/__init__.py:3:COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is
the seam: it returns a
backend/compute/__init__.py:16:from backend.infrastructure.execution_policy import
DEFAULT_POLICY, ExecutionPolicy
backend/compute/__init__.py:19:    "ComputeBackend", "ComputeJobSpec", "ComputeResult",
"get_compute_backend",
backend/compute/__init__.py:32:def get_compute_backend(
backend/compute/__init__.py:36:    policy: ExecutionPolicy = DEFAULT_POLICY,
backend/compute/__init__.py:42:    Only ``compute_target == "cloud_run"`` returns a non-None
backend (a ``CloudRunCompute``). Its
backend/compute/__init__.py:52:    from backend.compute.cloud_run import CloudRunCompute,
GoogleCloudRunJobClient
backend/compute/__init__.py:58:    return CloudRunCompute(
backend/compute/__init__.py:60:        storage_config=_storage_dict(config), policy=policy,
token=token,
```

23. assistant (2026-06-22T15:06:07.787Z)

The factory defaults to `DEFAULT_POLICY` (max_wait_sec=1800s = 30 min). So in the API path, a validation failure on the cloud worker would cause the bucket-poll to hang for the FULL max_wait budget (default 30 minutes, configurable) before raising `PolicyTimeout`, which is then caught by the broad `except` at main.py:512-514 and surfaced as a generic "cloud dispatch error".

Let me confirm the test file doesn't add a failure-marker path or otherwise contradict the finding, and check whether the API path was wired with a shorter policy.

24. user (2026-06-22T15:06:08.346Z)

```
59:+    fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend``
returns ``None``
61:+    from backend.compute import ComputeJobSpec, get_compute_backend
63:+    compute = get_compute_backend(global_config)
```

25. assistant (2026-06-22T15:06:12.694Z)

The API path calls `get_compute_backend(global_config)` with NO policy override, so it uses `DEFAULT_POLICY` (30-min budget). Confirmed.

Now let me verify the finding's claim about the test that exercises the rc!=0 branch ? to see whether the tests prove the rc!=0 gate is genuinely unreachable for real worker failures, or whether the tests inject a failure marker (which would mean the path is testable but the real worker never produces it).

26. user (2026-06-22T15:06:14.000Z)

```
42:    def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
44:        self.video_id, self.returncode = video_id, returncode
52:        if self.returncode == 0:
65:            json.dump({"video_id": self.video_id, "returncode": self.returncode,
143:    fake = _FakeCloudClient(returncode=1)
148:    assert "returncode 1" in out["message"]
```

27. user (2026-06-22T15:06:17.805Z)

```
        into the local bucket exactly where the real worker would, so the bucket-poll + ingest resolve."""

        def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
            self.calls: list[list[str]] = []
            self.video_id, self.returncode = video_id, returncode
            self.rows = rows if rows is not None else [(2.74, 6.91, 5, "winner"),
                                                       (8.74, 11.38, 3, "unforced_error")]

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append(list(argv))
            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
            done_uri = argv[argv.index("--done-uri") + 1]
            if self.returncode == 0:
                outputs = os.path.join(out_uri, "outputs", self.video_id)
                os.makedirs(outputs, exist_ok=True)
                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                    json.dump({"video_id": self.video_id, "segment_count": len(self.rows),
                               "status": "success"}, f)
                with open(os.path.join(outputs, "intervals.csv"), "w", newline="", encoding="utf-8") as f:
                    w = csv.writer(f)
                    w.writerow(["start", "end", "shot_count", "ending_reason"])
                    for s, e, sc, er in self.rows:
                        w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
            os.makedirs(os.path.dirname(done_uri), exist_ok=True)
            with open(done_uri, "w", encoding="utf-8") as f:
                json.dump({"video_id": self.video_id, "returncode": self.returncode,
                           "segment_count": len(self.rows), "status": "success"}, f)
            return "exec-1"


    @pytest.fixture
    def cloud_env(tmp_path, monkeypatch):
        """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the bucket read an
        identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for hash+validate."""
        db = Database(str(tmp_path / "t.db"))
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        cfg = AppConfig.model_validate({
            "deployment": {"compute_target": "cloud_run"},
            "storage": {"backend": "local", "output_root": str(bucket)},
        })
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        # A real gs:// input would be fetched to hash+validate; return a local fake so that path is offline.
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(m.storage, "acquire_local_input", lambda path, scfg: str(fake_local))
        monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        return db, bucket, monkeypatch


    def _inject_fake(monkeypatch, fake):
        """Make backend.compute.get_compute_backend (re-imported inside _maybe_dispatch_remote) build a
        real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed token."""
        real = compute_pkg.get_compute_backend
```

```
96          monkeypatch.setattr(compute_pkg, "get_compute_backend",
97                              lambda config: real(config, run_client=fake, policy=_FAST,
token="tok"))
98
99
100     def _meta(row):
101         md = row.get("metadata")
102         return json.loads(md) if isinstance(md, str) else (md or {})
103
104
105     def test_cloud_dispatch_ingests_segments(cloud_env):
106         db, _bucket, monkeypatch = cloud_env
107         fake = _FakeCloudClient()
108         _inject_fake(monkeypatch, fake)
109
110         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
111
112         assert out["status"] == "success"
113         assert len(out["play_segments"]) == 2
114         rows = db.query_play_segments(out["video_id"], {})
115         assert len(rows) == 2
116         metas = [_meta(r) for r in rows]
117         assert all(md["source"] == "cloud_run" for md in metas)
118         assert {md.get("shot_count") for md in metas} == {5, 3}
119         assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
120
121
122     def test_cloud_dispatch_builds_correct_spec(cloud_env):
123         _db, bucket, monkeypatch = cloud_env
124         fake = _FakeCloudClient()
125         _inject_fake(monkeypatch, fake)
126
127         m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
128
129         assert len(fake.calls) == 1  # exactly one Cloud Run Job execution
130         argv = fake.calls[0]
131         assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
132         assert argv[argv.index("--out-uri") + 1] == str(bucket)
133         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
134         joined = " ".join(argv)
135         assert "deployment.compute_target=local_gpu" in joined  # container runs INLINE
(never re-dispatch)
136         assert "indexing.detector_impl=native" in joined
137
138
139     def test_cloud_failure_marks_failed_no_rows(cloud_env):
140         db, _bucket, monkeypatch = cloud_env
141         # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
142         db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
143         fake = _FakeCloudClient(returncode=1)
144         _inject_fake(monkeypatch, fake)
145
146         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
147         assert out["status"] == "failed"
148         assert "returncode 1" in out["message"]
149         assert out["play_segments"] == []
150         assert db.query_play_segments(out["video_id"], {}) == []
151
152
153     def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
154         _db, _bucket, monkeypatch = cloud_env
```

```
155         fake = _FakeCloudClient()
156         _inject_fake(monkeypatch, fake)
157         local_path = str(tmp_path / "local_only.mp4")
158
159         out = m._execute_processing(m.ProcessRequest(video_path=local_path,
segmenter="wasb_hybrid"))
160         assert out["status"] == "failed"
161         assert "cloud input" in out["message"]
162         assert fake.calls == []  # never dispatched a local-only path
163
164
165     def test_default_off_runs_in_process(tmp_path, monkeypatch):
166         """compute_target='' ? get_compute_backend returns None ? the in-process segmenter
runs
167         byte-identically (the gate is inert). No Cloud Run dispatch."""
168         db = Database(str(tmp_path / "t.db"))
169         cfg = AppConfig()  # compute_target defaults to ""
```

28. assistant (2026-06-22T15:06:26.499Z)

This is the smoking gun that confirms the finding. Look closely at the test `_FakeCloudClient.run_job` (lines 48-67):

- The fake **ALWAYS writes the done-marker** (lines 63-66), **regardless of returncode**, with the injected `returncode` value.
- The test `test_cloud_failure_marks_failed_no_rows` (line 139-150) sets `returncode=1` and the fake STILL writes a marker with `returncode: 1`.

This is exactly the divergence the finding identifies: the test's fake worker writes a failure marker (with a non-zero returncode), but the **REAL worker** (`backend/worker/__main__.py`) does NOT ? it returns 2/3 BEFORE writing any marker (lines 200, 212), and only writes a marker on success with a hardcoded `0` (line 260).

So:
1. The rc!=0 gate at main.py:515-516 is tested ONLY against a fictional fake that writes failure markers the real worker never produces.
2. In production, a real worker validation/segmenter failure ? no marker ? `poll_until` hangs the full max_wait_sec (default 1800s) ? `PolicyTimeout` ? caught by the broad `except` at main.py:512-514 ? reported as generic "cloud dispatch error".

The finding is fully corroborated. Let me do a final check on one nuance the finding raises: whether `poll_until` for the API path is genuinely unbounded-ish (multi-minute) vs the test's `_FAST` policy.

29. user (2026-06-22T15:06:27.448Z)

```
1       """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3       Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4       intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5       GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6       done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7       dispatches; every other target runs the in-process segmenter byte-identically.
8       """
9       import csv
10      import json
11      import os
12
13      import pytest
14
15      pytest.importorskip("httpx")
```

```
16      from fastapi.testclient import TestClient
17
18      import backend.compute as compute_pkg
19      import backend.main as m
20      from backend.compute.cloud_run import CloudRunJobClient
21      from backend.config.models import AppConfig
22      from backend.infrastructure.database import Database
23      from backend.infrastructure.execution_policy import ExecutionPolicy
24      from backend.pipeline.validators.base import ValidationResult
25
26      _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
27
28
29      def _pass():
30          return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
31
32
33      class _InlineExecutor:
34          def submit(self, fn, *a, **k):
35              fn(*a, **k)
36
37
38      class _FakeCloudClient(CloudRunJobClient):
39          """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
```

30. assistant (2026-06-22T15:06:42.189Z)

Confirmed. The test uses `_FAST` policy (max_wait_sec=5.0), but production uses `DEFAULT_POLICY` (max_wait_sec=1800.0). And critically, the test's fake always writes a marker even on failure, while the real worker does not.

I have fully verified every load-bearing claim in the finding. Let me summarize the verification before submitting.

Every claim checks out:

1. ? main.py:515-516 ? `if result.returncode != 0: return _failed(...)` gate exists.
2. ? worker `__main__.py:259-260` ? done-marker written ONLY on success path, hardcoded `0` (`_write_done_marker(args.done_uri, video_id, 0, ...)`).
3. ? worker `__main__.py:200` ? validation failure returns 2 before marker.
4. ? worker `__main__.py:212` ? segmenter Failure returns 3 before marker.
5. ? The worker's own comment (lines 256-258) explicitly admits: "Written on SUCCESS only ? a failed container job currently lets the dispatcher's bounded poll time out (a failure-marker is a follow-up)."
6. ? cloud_run.py:119 ? `run_infer` bucket-polls via `poll_until`; on no-marker it raises `PolicyTimeout` (execution_policy.py:147).
7. ? main.py:511 calls `compute.run_infer(spec)` DIRECTLY (not via worker's `_dispatch_remote`), so the rc=4 conversion in `_dispatch_remote` (worker `__main__.py:134-136`) does NOT apply to the API path. The `PolicyTimeout` is caught by the broad `except Exception` at main.py:512-514 ? generic "cloud dispatch error".
8. ? base.py:36-38 docstring claims "4 = remote dispatch timeout" ? but rc=4 is produced ONLY in the worker CLI path, never in the API path (`ComputeResult.returncode` from `run_infer` is the marker's rc, which is only ever 0 for real success).
9. ? API path uses `DEFAULT_POLICY` (max_wait_sec=1800s = 30 min) ? the "multi-minute hang" is real (default 30 min).
10. ? The test `test_cloud_failure_marks_failed_no_rows` only passes because the FAKE client writes a failure marker (returncode=1) the real worker never produces ? confirming the rc!=0 gate is effectively dead against real worker failures.

One small refinement: the finding says rc=4 "is never produced." That is true for the **API path** (the finding's scope), but rc=4 IS produced in the worker CLI dispatch path (`_dispatch_remote`). The finding's core claim ("the explicit returncode!=0 handling is unreachable for genuine failures" in main.py, and the rc=4 docstring path doesn't exist *on the cloud path the API uses*) holds. This is a minor imprecision that doesn't undermine the defect.

This is a real dispatch-correctness defect. Severity major is justified: a bad-video on the cloud path costs a 30-minute hang plus an opaque error instead of a fast accurate failure.

31. user (2026-06-22T15:06:54.091Z)

Structured output provided successfully

32. assistant (2026-06-22T15:07:05.473Z)

Verified. This is a real dispatch-correctness defect (major). The finding's core claims all hold against the cited code on branch commit 304252b in `C:/Users/avidu/Projects/bhi-cloudrun`:

- `backend/main.py:515-516` ? the `result.returncode != 0` gate is real and reachable only via the test's fake, never via a real worker failure.
- `backend/worker/__main__.py` ? validation (`return 2`, line 200) and segmenter (`return 3`, line 212) failures both return BEFORE any marker; the marker is written only on success (lines 259-260) with a hardcoded `0`. The code comment itself admits this gap.
- `backend/compute/cloud_run.py:119` polls via `poll_until`, which raises `PolicyTimeout` (no rc=4 conversion on the API path); main.py:512-514 swallows it as a generic "cloud dispatch error" after the full `DEFAULT_POLICY` budget (`max_wait_sec=1800s` = 30 min).
- `backend/compute/base.py:36-38` documents an rc=4 path that the API/cloud path never produces.
- The new test `tests/test_api_cloud_dispatch.py` `_FakeCloudClient` writes a failure marker the real worker never emits, which is precisely why the dead branch appears covered.

One minor imprecision in the finding (rc=4 IS produced in the worker CLI `_dispatch_remote` path, just not on the API path it scopes to) does not undermine the defect or the recommended fix.